

Systemdesign

Systemdesign

Während das Softwaredesign sich hauptsächlich mit einzelnen Komponenten und deren Design/Umsetzung beschäftigt, geht es beim Systemdesign darum, verschiedene einzelne Komponenten (Services) zu einer großen Applikation zusammenzufügen.

Wir werden uns mit dem grundsätzlichen Aufbau eines Systemdesigns befassen, und mit verschiedenen gängigen Komponenten.

In diesem Kapitel werden wir nicht auf einzelne Klassen/Module eingehen, sondern lediglich auf deren Interaktion.

Inhalt

- Systemdesign
 - Erstellen des Systemdesigns
 - Sequenzdiagramme
 - API
- Systemkomponenten
 - Datenbank
 - Caches
 - Microservices

Systemdesign

Ein Systemdesign gibt eine Übersicht über alle relevanten Komponenten, die in einem System zusammenspielen, und deren Schnittstellen.

Es gibt keine Hinweise auf die Art und Weise, wie die Komponenten im Detail funktionieren.

In einem Systemdesign geben wir einen möglichst kompakten Überblick über alle Komponenten des Systems. Hierbei sollten, wenn möglich, Diagramme und kein Text verwendet werden.

Insbesondere wird ein Systemdesign noch vor dem Komponentendesign erzeugt. Es dient als Leitlinie für alle weiteren Prozesse.

Beispiel: Erzeuge einen tinyURL Service

Wir wollen einen tinyURL Service designen. Hierbei kann ein Nutzer eine URL eingeben, und eine kurze URL wird zurückgegeben.

Wenn man dann die kurze URL aufruft, wird man auf die ursprüngliche URL weitergeleitet.

Beispiel: www.google.com -> tinyurl.com/random

tinyURL: Klärung der Spezifikation

Am Anfang des Systemdesigns sollten so lange Fragen zur Funktionalität gestellt werden, bis die wesentliche Funktionalität des Systems geklärt ist. Diese können sowohl aus Sicht des Nutzers gestellt werden, als auch aus Sicht des Entwicklers. In unserem Fall könnten wir folgende Fragen stellen:

- Wie viele URLs soll das System unterstützen?
- Sollen die tinyURLs auf die langen URLs schließen lassen?
- Wie lange soll eine tinyURL existieren?

Manche benötigten Informationen werden erst während des Designs sichtbar.

Fragen und Antworten

- Wie viele URLs soll das System unterstützen?
 - Das System soll beliebig viele URLs unterstützen, ein Minimum sollten 10 Millionen sein.
- Sollen die tinyURLs auf die langen URLs schließen lassen?
 - Nein
- Wie lange soll eine tinyURL existieren?
 - Eine URL muss mindestens 30 Tage nach dem letzten Zugriff existieren.

Annahmen

Basierend auf der Problemstellung und den Fragen formulieren wir Annahmen, unter welchen wir das System designen.

- Das System erzeugt eine zufällige tinyURL
- Das System speichert die Weiterleitungen für immer
- Das System unterstützt mindestens 10 Millionen Nutzer

Weiterhin definieren wir die nötigen Interaktionen

- Nutzer sollen Weiterleitungen erzeugen und löschen können
- Weiterleitung soll automatisch über den Browser erfolgen

APIs

Als erstes definieren wir das User Interface. Wir haben bereits festgelegt, dass das Abfragen der Weiterleitung direkt über die URL erfolgen soll.

Somit müssen wir nur festlegen, wie eine URL erstellt und wie gelöscht werden soll.

Die Festlegung einer solchen API erfolgt normalerweise über Funktionen. Wie diese später aufgerufen werden sollen, ist irrelevant. Lediglich die Argumente und der Rückgabewert spielen eine Rolle. Wir können also eine funktionale Definition verwenden.

Hinzufügen der URL

Wir definieren die API für das Hinzufügen der URL als

```
add_url: string -> bool x string
```

Die Eingabe ist die ursprüngliche URL. Die Rückgabe ist ein Boolean, der Erfolg oder Misserfolg signalisiert, und, im Falle des Erfolgs, die gekürzte URL, oder im Falle eines Misserfolgs, eine Fehlermeldung.

Beispiel: Hinzufügen der URL

Der Aufruf von `add_url('www.th-owl.de')` kann also folgende Werte zurückgeben:

- Erfolg: (True, 'tinyurl.com/abcdef')
- Misserfolg: (False, 'Service unavailable')

Der Nutzer des Services muss dann selbst entscheiden, was zu tun ist.

Normalerweise gibt man keine Booleans, sondern so genannte error codes zurück. Diese definieren die Art des Fehlers.

Löschen einer Weiterleitung

Wir definieren die API für das Löschen einer Weiterleitung als

`delete_url: string -> bool.`

Die Eingabe ist hier die gekürzte URL, die Ausgabe eine Indikation über Erfolg oder Misserfolg. Auch hier können wir wieder eine Fehlermeldung oder einen Fehlercode zusätzlich zurückgeben.

Wir benutzen die gekürzte URL und nicht die Ursprungs-URL, um die Möglichkeit zu unterstützen, dass es verschiedene unabhängige Weiterleitungen auf dieselbe URL gibt.

API best practices

Beim Erstellen der API sollten folgende Dinge beachtet werden:

- Die Funktionen der API sollten im Zusammenhang des Systems direkt klar werden. Gute Namen sind hierbei das Wichtigste.
- Die Argumente sollten intuitiv sein. Wenn möglich, sollte auf Tupel oder Dictionaries zurückgegriffen werden. Dies ermöglicht auch größere Flexibilität.
- Es gibt verschiedene API-Techniken (REST, HTTP, Header, etc.). Manchmal ist es hilfreich, den gewünschten API-Typ bereits zu kennen.

Speichern der URLs

Um die verschiedenen Weiterleitungen zu speichern, wollen wir eine Datenbank verwenden. Eine Datenbank ist ein persistenter Speicher, in dem wir suchen können. Es gibt verschiedene Typen von Datenbanken, auf die wir nicht weiter eingehen möchten.

Wichtig für uns ist:

- In einer Datenbank können wir speichern und suchen
- Die Datenbank schützt uns vor race conditions
- Die Datenbank ist ein sicherer Speicher

Heutzutage werden größtenteils Datenbanken als Services benutzt (Cloud Datastore, DynamoDB). Selbstverwaltete Datenbanken sind teuer, und werden nur dann verwendet, wenn dies aus technischen Gründen notwendig ist.

Datenbank Design

Normalerweise werden Daten in Datenbanken in der Form von Tabellen angelegt. Moderne Datenbanken können nach sämtlichen Indices durchsucht werden. Es empfiehlt sich jedoch, einen Eintrag als Schlüssel festzulegen, nach dem Sortiert wird, um einen schnellen Zugriff zu haben (ähnlich wie beim Dictionary).

Unsere Datenbank benötigt zwei Felder in der Tabelle URLs:

ShortenedURL: (str, key)	RedirectionURL: str
--------------------------	---------------------

key deutet an, dass die Datenbank nach diesem Feld sortiert werden soll.

Datenbanken

Normalerweise werden Datenbanken über SQL, der “Structured Query Language” angesprochen. In SQL könnte eine Abfrage für eine gekürzte URL so aussehen:

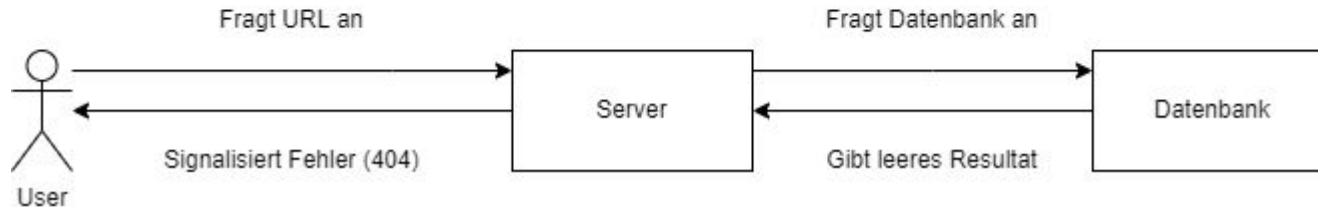
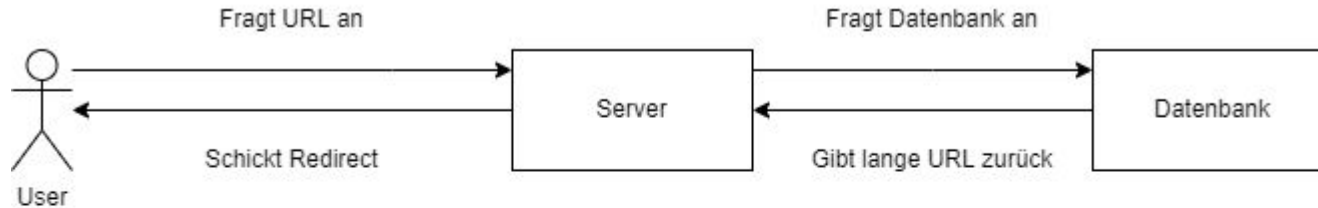
```
SELECT RedirectionURL FROM URLs WHERE ShortenedURL = '1234'
```

Wir wollen in dieser Vorlesung nicht weiter auf SQL eingehen. Jedoch sollte man sich beim Erstellen der Datenbank bereits Gedanken darüber machen, welche Queries man ausführen möchte, um festzustellen, wie man die Tabellen in der Datenbank ablegt.

Designdiagramme

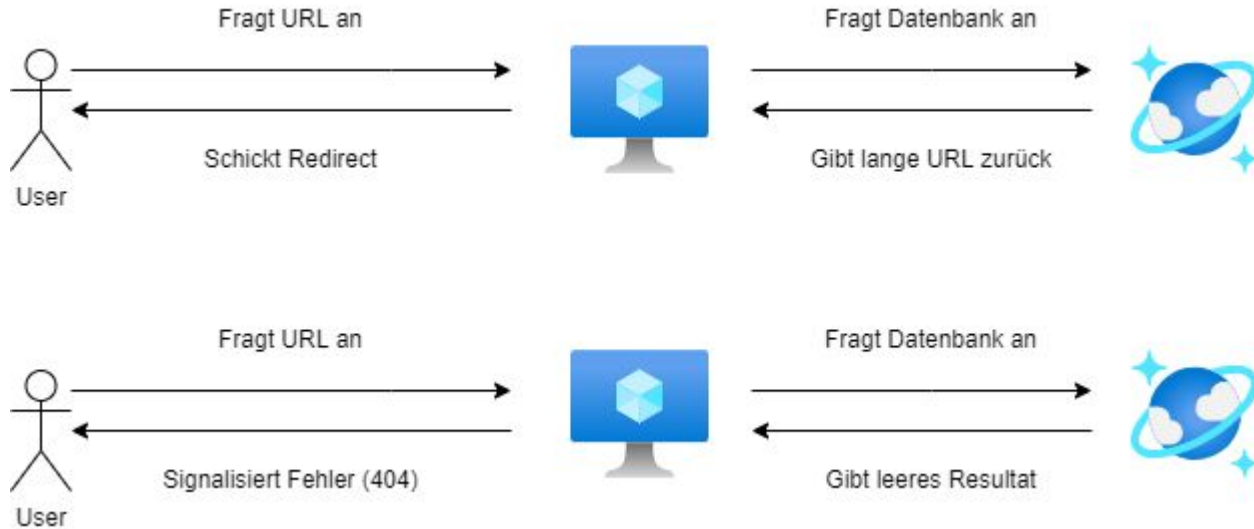
Nachdem wir die einzelnen Komponenten (Datenbank, API) definiert haben, geben wir die Funktionalität in einzelnen Diagrammen an.

Wir beginnen mit dem Diagramm für einen Zugriff auf eine gekürzte URL



Designdiagramme

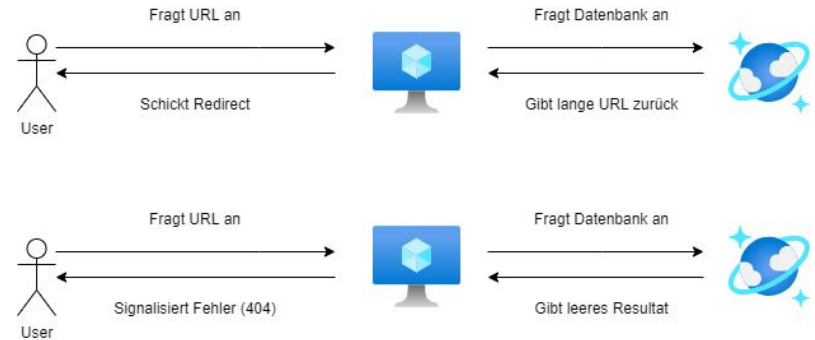
Da oft direkt die Services des jeweils bevorzugten Cloud-Anbieters verwendet werden, würde ein entsprechendes Design in Microsoft Azure so aussehen:



Cloud Services

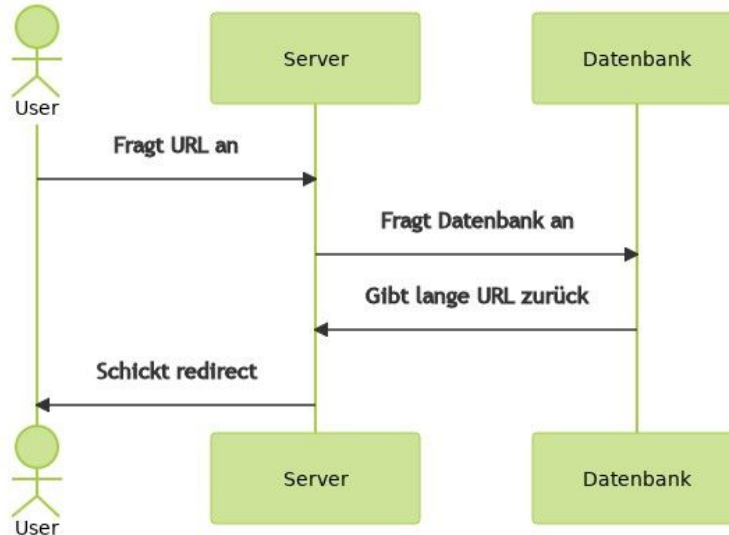
Die Menge an angebotenen Cloud Services variiert stark nach Anbieter, viele oft verwendete Services werden jedoch von fast allen Anwendern angeboten.

Es empfiehlt sich, eine kleine Übersicht zu haben (Datenbanken, cloud compute, cloud storage, load balancer, etc.)



Alternatives Diagramm: Sequenzdiagramm

Als eine Alternative zum vorher gegebenen Diagramm kann ein sogenanntes Sequenzdiagramm angegeben werden. Es verdeutlicht den zeitlichen Ablauf besser:



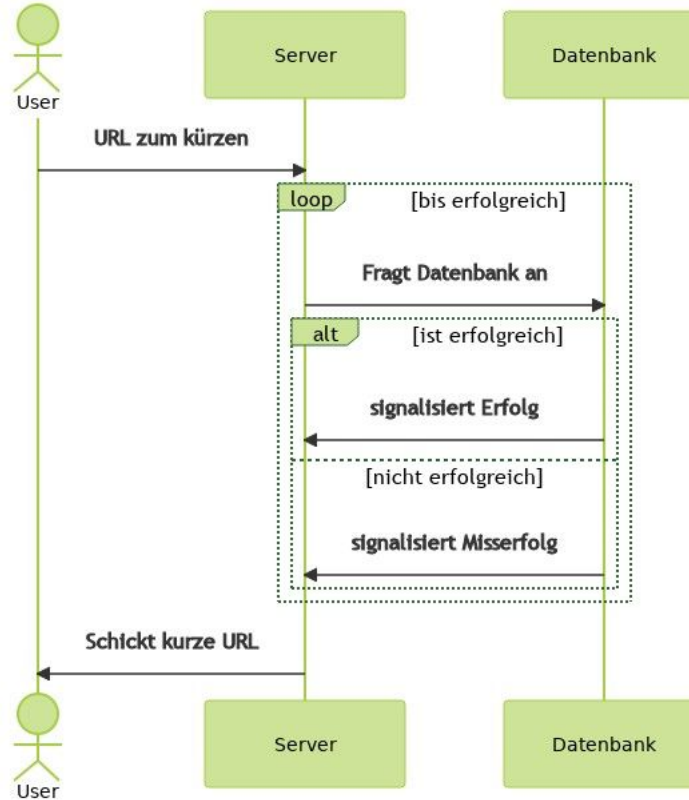
Erstellen einer URL

Nun wollen wir ein Diagramm zum erstellen einer URL erzeugen.

Wir arbeiten unter der Voraussetzung, dass wir so lange zufällige gekürzte URLs erzeugen, bis wir eine URL finden, die noch nicht in der Datenbank vertreten ist.

Hiervon soll der Nutzer nichts merken. Er soll immer eine gekürzte URL erhalten, solange die Datenbank nicht voll ist, was von der Datenbank signalisiert wird.

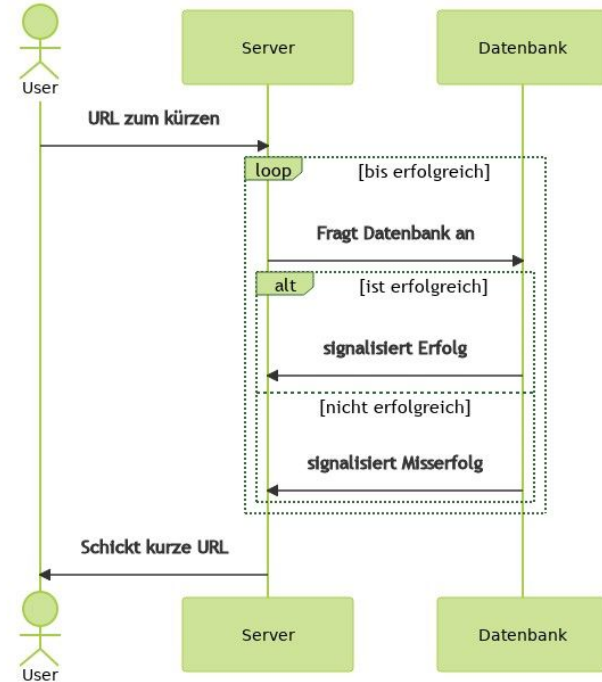
Sequenzdiagramm



Sequenzdiagramm

Im Sequenzdiagramm können verschiedene Möglichkeiten der wiederholten oder optionalen Kommunikation angegeben werden.

- loop steht für eine Schleife
- alt für alternative Kommunikation
- par für parallele Kommunikation
- opt für optionale Kommunikation



Sequenzdiagramme mit Mermaid

Sequenzdiagramme können mit draw.io oder mit [Mermaid](#) erzeugt werden.

```
sequenceDiagram
    actor User
    participant Server
    participant Datenbank
    User ->> Server: URL zum kürzen
    loop bis erfolgreich
        Server ->> Datenbank: Fragt Datenbank an
        alt ist erfolgreich
            Datenbank ->> Server: signalisiert Erfolg
        else nicht erfolgreich
            Datenbank ->> Server: signalisiert Misserfolg
        end
    end
    end
    Server ->> User: Schickt kurze URL
```


SQL Query

Das Einfügen eines Elementes in die Datenbank kann mit folgendem SQL Befehl passieren:

```
INSERT INTO URLs (ShortenedURL, RedirectionURL) VALUES ('a', 'b')
```

Wir teilen der Datenbank mit, in welche Tabelle wir für welche Spalten welche Werte einfügen wollen.

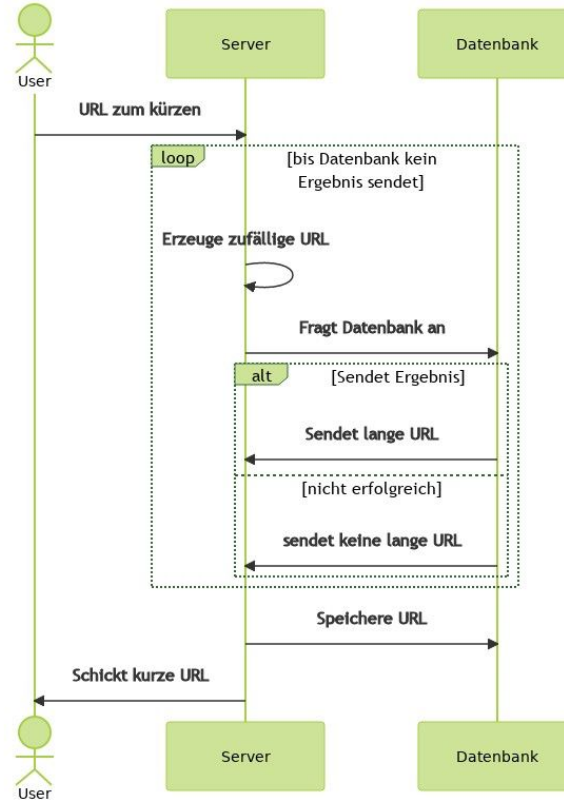
Alternative Version: Einfügen einer gekürzten URL

Wir sind in der vorherigen Version des Ablaufs davon ausgegangen, dass die Datenbank uns signalisiert, ob das Einfügen geklappt hat. Diese Funktionalität ist nicht in jeder Datenbank gegeben.

Wir können das Design jedoch auch so anlegen, dass wir in der Datenbank nachschauen, ob der Eintrag bereits existiert, und wenn nicht, den neuen Eintrag einfügen.

Dies können wir dann ebenfalls im Sequenzdiagramm vermerken.

Sequenzdiagramm

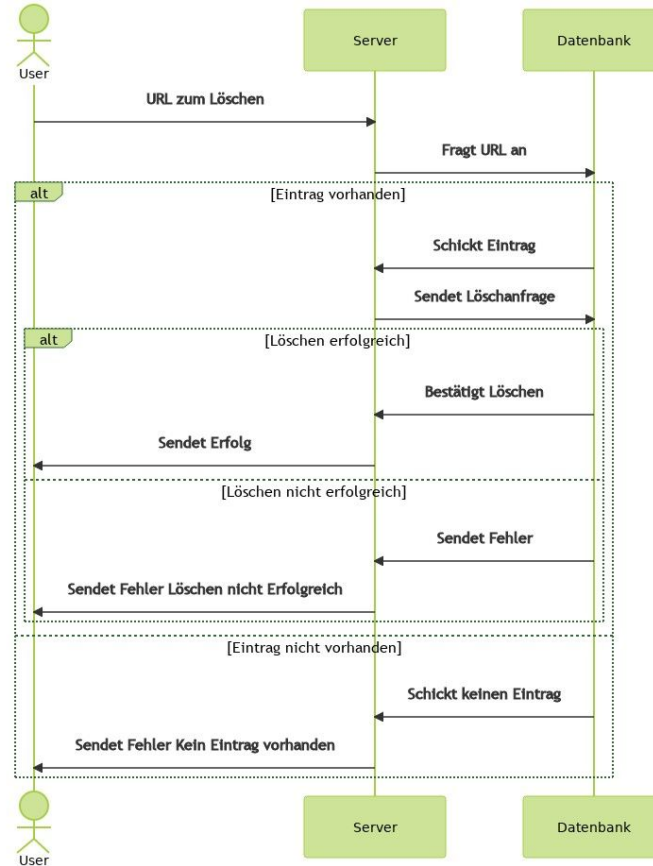


Löschen einer URL

Das Löschen einer URL ist eine Löschanfrage an den Server. Diese kann entweder Erfolgreich oder nicht Erfolgreich sein. Ein Nichterfolg kann stattfinden, wenn die URL nicht in der Datenbank ist, oder wenn das Löschen nicht möglich ist.

Wir erstellen wieder ein Sequenzdiagramm.

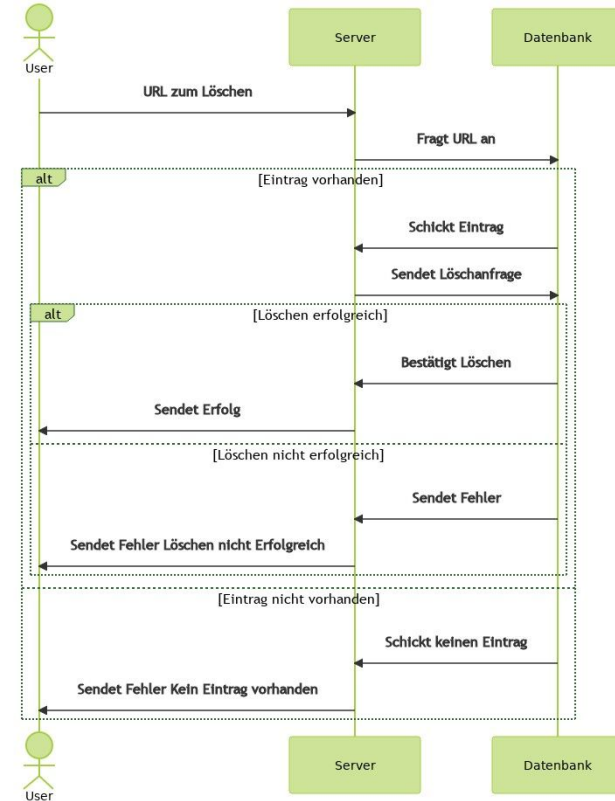
Sequenzdiagramm



Sequenzdiagramm: Alternative

Alternativ wäre es hier möglich, einen Löschantrag eines nicht vorhandenen Eintrags als Erfolg zurück zu geben.

Diese Alternativen können beide in das Design aufgenommen werden, um dann abgesprochen zu werden.



SQL Query

Das Löschen eines Elementes kann mit dem folgenden SQL Befehl vorgenommen werden:

```
DELETE FROM URLs WHERE ShortenedURL = '1234'
```

Wir teilen so der Datenbank mit, aus welcher Tabelle wir welchen Wert löschen wollen.

Gesamtes Design

Unser gesamtes Design besteht nun aus:

- Annahmen für das Design
- APIs
- Datenbank
- Sequenzdiagrammen

Mit Hilfe dieser Aufzeichnungen kann das Design besprochen werden und anschließend zum Design der Komponenten (OOP, Funktional) übergegangen werden.

Weitere Technik: Cache

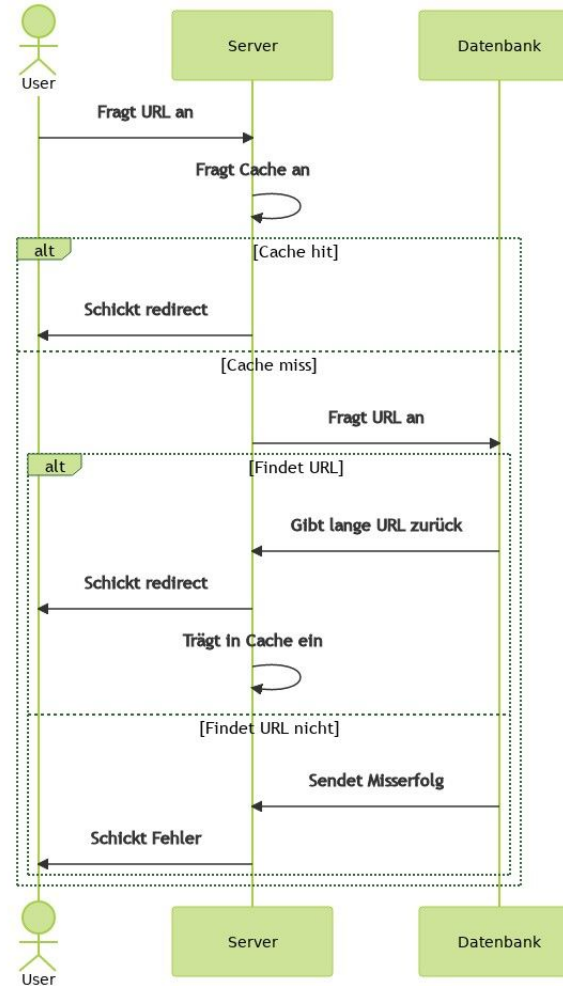
Um die Zugriffe auf die Datenbank zu reduzieren (gerade bei größeren Datenbanken kann dies zu größeren Verzögerungen führen) können wir den Server mit einem LRU (least recently used) Cache ausstatten.

Dieser Cache enthält immer die X letzten Aufrufe für den Server.

In unserem Fall würde der Cache also für die letzten X kurzen URLs die passende lange URL enthalten.

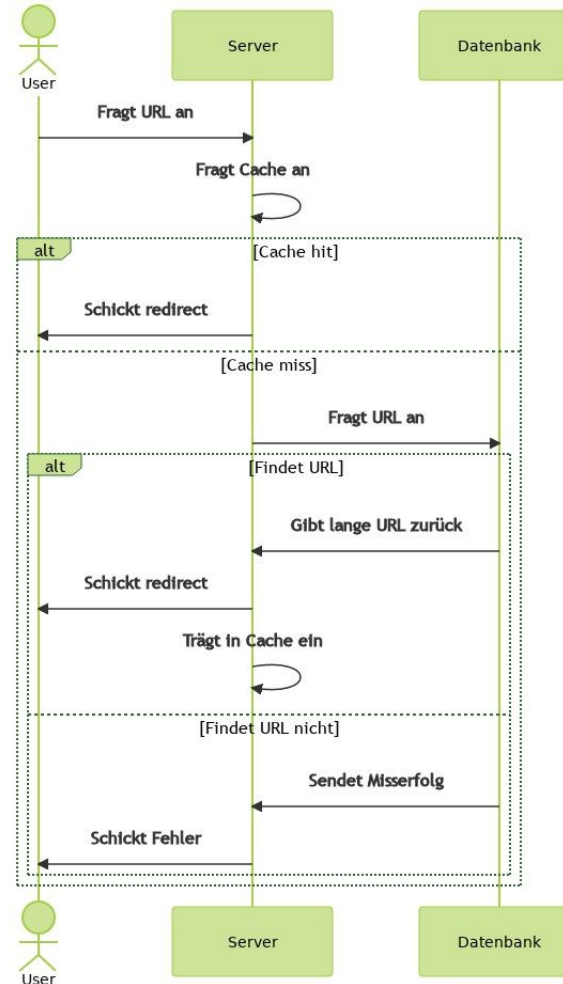
Eine solche Erweiterung kann als Alternative in das Design mit aufgenommen werden.

Sequenzdiagramm



Sequenzdiagramm

Ein Problem ist nun, dass wir auch ein neues Diagramm für das Löschen einer URL brauchen, denn diese muss aus dem Cache entfernt werden (Übung).



REST APIs

Im bisherigen Design haben wir unsere APIs durch Funktionen definiert. Dies hilft, den allgemeinen Ablauf eines Programms mit verschiedenen Komponenten zu beschreiben.

Ein tinyURL service würde jedoch ein über den Browser ansprechbares Interface benötigen. Man verwendet hier oft so genannte REST APIs (Representational State Transfer Application Programming Interface). Man benutzt für die Interaktion mit dem System einen HTTP endpoint, zusammen mit einem Pfad (API Pfad), und einer Funktion (GET, POST, DELETE, PUT, PATCH).

Desweiteren beschreibt man, welche Daten von und zum Server gesendet werden.

REST APIs: Hinzufügen der URL

Die Definition des Pfads könnte so aussehen:

POST /create-url

Hierbei bedeutet POST, dass man eine neue Ressource (Weiterleitung erzeugt). create-url ist der Pfad, mit dem die passende Funktion auf dem Server angesprochen wird, zum Beispiel: <https://my-server.org/create-url>.

REST APIs: Hinzufügen der URL

Zum Beschreiben der Daten, die an den Endpoint gesendet werden, benutzt man JSON (Javascript Object Notation) schemas. Das JSON schema zum Senden der URL, die gekürzt werden soll, kann so aussehen.

Man sieht hier, dass in der Message nur das Feld url benötigt wird.

```
{
  "$schema":
    "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "url": {
      "type": "string",
      "format": "uri",
      "description": "The original long URL to be
shortened"
    }
  },
  "required": ["url"],
  "additionalProperties": false
}
```

Vom Monolith zum Microservice

In unserem bisherigen Design haben wir einen Webserver verwendet, also eine monolithische Architektur erzeugt. Dies hat Vorteile:

- Unsere Applikation kann mit einem Befehl gestartet werden
- Es gibt schnellere Kommunikation zwischen den einzelnen Komponenten
- Schnelle und einfache Entwicklung

Lediglich die Datenbank haben wir als einzelne Komponente angegeben, wir hätten diese jedoch auch in die App integrieren können.

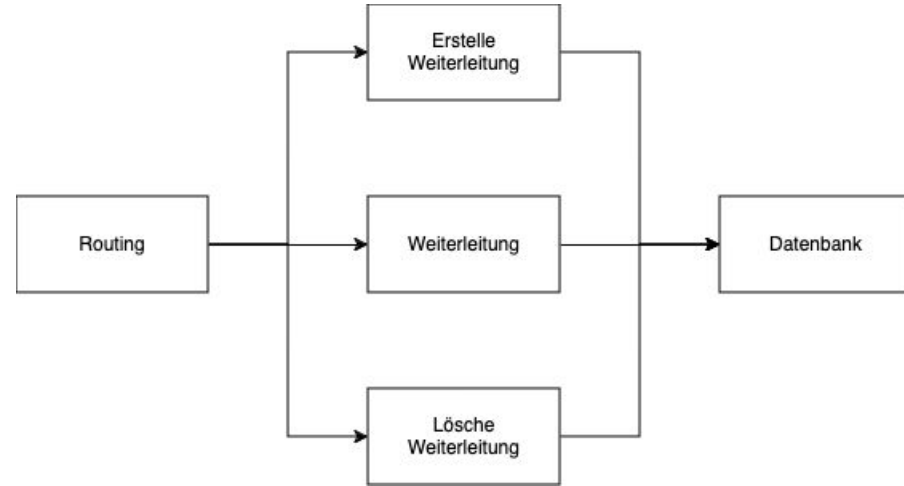
Das Problem in einer monolithischen Architektur ist die Skalierung. Wenn ein einzelner Computer nicht mehr ausreicht, um alle Anfragen schnell zu bearbeiten, benötigt man weitere Server, einen load balancer, und muss aufpassen, dass Daten zwischen den Servern konsistent sind.

Vom Monolith zum Microservice

Um die angesprochenen Probleme zu vermeiden, werden viele Webapplikationen heutzutage direkt als Microservices konstruiert. Dies bedeutet, dass jede Funktion in einer eigenen Applikation bereitgestellt wird.

In unserem Fall haben wir drei Funktionen: Weiterleitung erstellen, Weiterleitung anfragen, und Weiterleitung löschen. Diese können unabhängig voneinander mit der Datenbank kommunizieren.

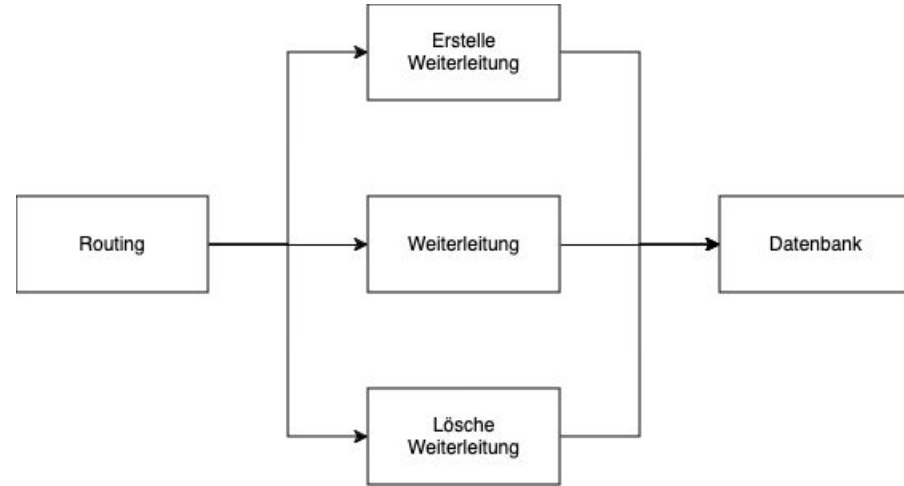
Im Design haben wir nun drei Server anstatt einem.



Vom Monolith zum Microservice

Das Microservice Design bringt mehrere Vorteile:

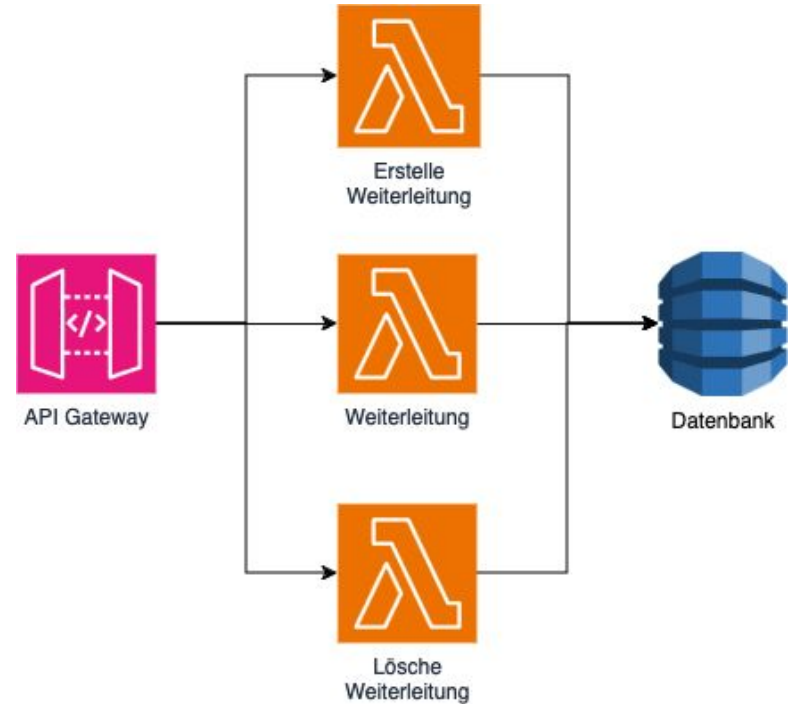
- Komponenten können unabhängig skaliert werden. Da Weiterleitung vermutlich am meisten verwendet wird, kann diese Komponente mit einem load balancer ausgestattet werden, und mehrere Computer verwenden.
- Einzelne Komponenten haben ihre eigenen APIs. Dementsprechend können sie von unterschiedlichen Teams betreut werden.



Microservice: Serverless design

Ein weiterer Vorteil von Microservices ist, dass sie in den “managed compute” oder “serverless design” der größeren Cloud provider fallen.

Anstatt Server direkt zu mieten, benutzt man hier vorgefertigte Komponenten, die nicht verwaltet werden müssen. Probleme mit Skalierung oder Update des Betriebssystems fallen weg.



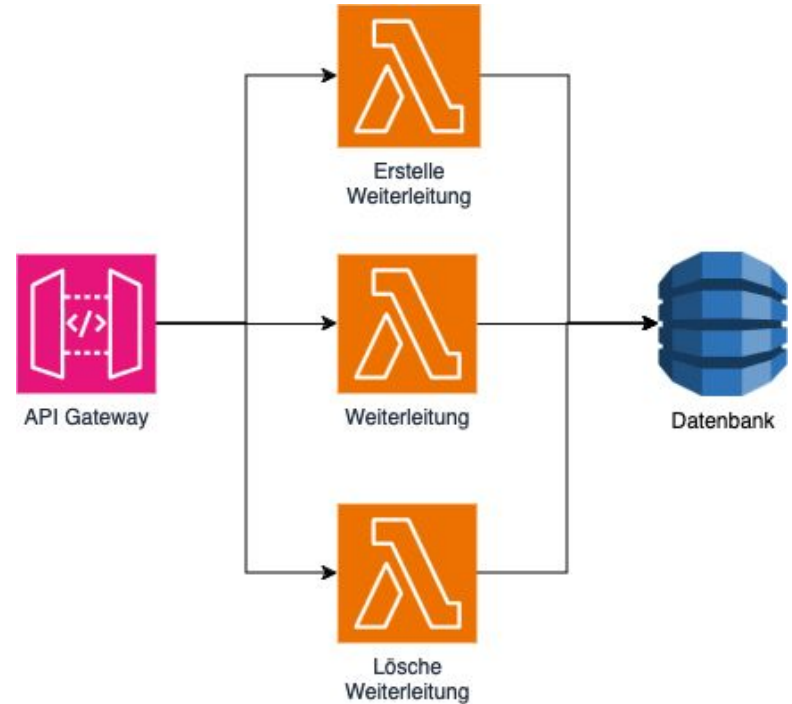
Microservice: Serverless design

In diesem Beispiel werden drei Komponenten des AWS Stacks benutzt: API Gateway, Lambda, und DynamoDB.

DynamoDB ist eine verwaltete Datenbank, in der man einzelne Tabellen anlegen kann.

Lambda sind einzelne Funktionen, die gestartet werden, wenn man eine Anfrage an sie sendet, und die dann das Ergebnis zurückliefern. Man benötigt keinen Servercode oder eine Webapplikation. Lediglich der Funktionscode muss bereitgestellt werden.

API Gateway erzeugt eine URL, für welche man REST Methoden definieren kann, welche dann wiederum die Lambdas aufrufen.



Microservice: Serverless design

Serverless design bietet sich insbesondere an, wenn man keine Infrastruktur verwalten möchte. So können komplexe Webapplikationen von wenigen Entwicklern betreut werden.

Die meisten verwalteten Lösungen der Cloudprovider werden außerdem nach Benutzung bezahlt, z.B. API Gateway pro Anfrage, Lambda pro Millisekunde. Dies kann günstig sein, wenn eine Webapplikation nur wenige Benutzer hat. Ein voll ausgelasteter Server ist jedoch im allgemeinen billiger. Auch hier kommt es auf die Komplexität der Applikation an.